These repositories also come with features to ensure that two or more changes that do not match with each other are not allowed in, thus preventing the accidental overwriting of one developer's code by another. Source code repositories also often feature integration modules with IDEs and deployment tools to allow seamless inflow and outflow of code. While writing to a source code repository might seem simple, there are a few standard practices to follow. Some examples of popular source code repositories are Git, Subversion, Mercurial etc.

If it's not in source control, it doesn't exist

There is a common mantra that says: "The only measure of progress is working code in source control". As established earlier, source repositories are the one true source of code truth and unless the code you write is not part of it – it simply doesn't exist.

Commit early, commit often and don't spare the horses

Extending the previous point, the surest way to avoid "ghost code" – that which only you can see on your local machine – is to get it into the repository as early and as often as possible. The early and often approach achieves a few other things as well, which can make a significant difference:

1. Every committed revision gives you a rollback marker. If anything goes wrong, are you rolling back one hour worth of changes or one week?
2. The risk of a merge conflict nightmare increases dramatically with time. When you've not committed code for days and you suddenly realise you've got 50 conflicts with other people's changes, it won't be cool.

When you work this way, your commit history starts to resemble a regular pattern of multiple commits each work day. Of course it's not always going to be the same but the overall objective is achieved. However, when in an entire project there are entire days or even multiple days where nothing is happening, it might be reason to get worried. Often development is happening in a very "do-it-all" sort of way or absolutely nothing of value is happening at all because developers are stuck somewhere. Either way, something is wrong and source control can be a very useful indicator to let you know.

Always check your changes

Committing code into source control is easy but do too much of it carelessly and what you end up with is changes and files being committed without care!